

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Створення інтегрованих середовищ розробки»

Робота №2

Тема «Синтаксичний аналізатор»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Дейнега Л.Ю.

2025

МЕТА

Метою роботи є ознайомитись з основними поняттями, методами та засобами синтаксичного аналізу вихідної програми; навчитися проєктувати ат розробляти синтаксичний аналізатор у складі компілятора за допомогою генератора парсерів ANTLR.

ЗАВДАННЯ

Завдання на роботу:

1. Ознайомитись з теоретичними відомостями, необхідними для виконання роботи.

2. Визначити граматику для виконання синтаксичного аналізу програм мовою, заданою у відповідності з індивідуальним завданням з додатку А, узгодженим з викладачем.

Розробити IDE для мови програмування Rust. Обов'язково мають бути проаналізовані наступні можливості даної мови: вбудовані типи даних, функції, вектори, записи, методи, цикли, умовні оператори, зв'язування змінних, арифметичні оператори.

3. Розробити програмне забезпечення за допомогою генератора ANTLR мовою програмування Java на основі розробленого лексичного аналізатора, яке відповідає наступним вимогам:

- виконує синтаксичний аналіз тексту програми у відповідності з індивідуальним завданням;
- формує в результаті аналізу абстрактні синтаксичні дерева;
- виводить повідомлення про помилки, виявлені під час синтаксичного аналізу;

- має графічний інтерфейс та дозволяє виводити результати синтаксичного аналізу (передбачати також можливість роботи без графічного інтерфейсу).

4. Виконати тестування побудованого синтаксичного аналізатора.
5. Оформити звіт.
6. Відповісти на контрольні питання.

ВИКОНАННЯ

Код

Java

```
import gen.MainLexer;
import gen.MainParser;
import org.antlr.v4.runtime.*;
import org.antlr.v4.runtime.tree.ParseTree;
import org.antlr.v4.runtime.tree.Trees;

import javax.swing.*;
import javax.swing.tree.DefaultMutableTreeNode;
import javax.swing.tree.DefaultTreeModel;
import java.awt.*;

public class Main extends JFrame {
    private JTextArea codeArea;
    private JTree astTree;
    private JTextArea errorConsole;

    public Main() {
        setTitle("Rust Simple IDE (ANTLR4)");
        setSize(1000, 700);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        codeArea = new JTextArea();
        codeArea.setFont(new Font("Consolas", Font.PLAIN, 14));
        JScrollPane codeScroll = new JScrollPane(codeArea);
        codeScroll.setBorder(BorderFactory.createTitledBorder("Rust Source Code"));

        astTree = new JTree(new DefaultMutableTreeNode("AST Root"));
        JScrollPane treeScroll = new JScrollPane(astTree);
        treeScroll.setBorder(BorderFactory.createTitledBorder("Abstract Syntax Tree"));

        errorConsole = new JTextArea(8, 0);
        errorConsole.setEditable(false);
```

```

        errorConsole.setForeground(Color.RED);
        JScrollPane errorScroll = new JScrollPane(errorConsole);
        errorScroll.setBorder(BorderFactory.createTitledBorder("Error
Console"));

        JSplitPane mainSplit = new JSplitPane(JSplitPane.HORIZONTAL_SPLIT,
codeScroll, treeScroll);
        mainSplit.setDividerLocation(500);
        add(mainSplit, BorderLayout.CENTER);
        add(errorScroll, BorderLayout.SOUTH);

        JButton runBtn = new JButton("Analyze Code");
        runBtn.addActionListener(event -> runAnalysis());
        add(runBtn, BorderLayout.NORTH);
    }

    private void runAnalysis() {
        String code = codeArea.getText();
        errorConsole.setText("");

        try {
            CodePointCharStream input = CharStreams.fromString(code);
            MainLexer lexer = new MainLexer(input);
            CommonTokenStream tokens = new CommonTokenStream(lexer);
            MainParser parser = new MainParser(tokens);

            parser.removeErrorListeners();
            parser.addErrorListener(new BaseErrorListener() {
                @Override
                public void syntaxError(Recognizer, ? recognizer, Object
offendingSymbol, int line,
                                     int charPositionInLine, String msg,
RecognitionException e) {
                    errorConsole.append("Error at line " + line + ":" +
charPositionInLine + " - "+msg+"\n");
                }
            });

            ParseTree tree = parser.program();

            DefaultMutableTreeNode root = buildJTree(tree, parser);
            astTree.setModel(new DefaultTreeModel(root));

            if (errorConsole.getText().isEmpty()) {
                errorConsole.setBackground(new Color(0,207,0));
                errorConsole.setForeground(Color.black);
                errorConsole.setText("Analysis successful! No errors found.");
            } else {
                errorConsole.setBackground(new Color(207, 0, 0));
                errorConsole.setForeground(Color.RED);
            }
        } catch (Exception e) {
            errorConsole.append("System error: "+e.getMessage());
        }
    }
}

```

```

    private DefaultMutableTreeNode buildJTree(ParseTree antlrTree, MainParser
parser) {
        String text = Trees.getNodeText(antlrTree, parser);
        DefaultMutableTreeNode node = new DefaultMutableTreeNode(text);
        for (int i = 0; i < antlrTree.getChildCount(); i++) {
            node.add(buildJTree(antlrTree.getChild(i), parser));
        }
        return node;
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(()->new Main().setVisible(true));
    }
}

```

Antlr

grammar Main;

```

program: item* EOF;
item: function | structDef | implBlock;
function: 'fn' ID '(' paramList? ')' ('->' type_)? block;
paramList: param (',' param)*;
param: ID ':' type_;
type_: primitiveType | 'Vec' '<' type_ '>' | ID;
primitiveType: 'i32' | 'bool' | 'String';
structDef: 'struct' ID '{' fieldList? '}';
fieldList: field (',' field)*;
field: ID ':' type_;
implBlock: 'impl' ID '{' method* '}';
method: function;
block: '{' statement* '}';
statement: letStmt | expr ';' | ifStmt | forLoop | whileLoop | ';';
letStmt: 'let' ID ':' type_? '=' expr ';';
ifStmt: 'if' expr block ('else' (block | ifStmt))?;
forLoop: 'for' ID 'in' expr '..' expr block;
whileLoop: 'while' expr block;
expr
    : expr binOp expr          # binaryExpr
    | unOp expr                # unaryExpr
    | ID '(' exprList? ')'      # callExpr
    | expr '.' ID '(' exprList? ')' # methodCallExpr
    | expr '.' ID               # fieldAccess
    | 'vec!' '[' exprList? ']'   # vecLiteral
    | literal                   # litExpr
    | ID                        # varExpr
    | '(' expr ')'              # parenExpr
    ;
exprList: expr (',' expr)*;
literal: INT | STRING | 'true' | 'false';
binOp: '+' | '-' | '*' | '/' | '%' | '==' | '!=' | '<' | '>' | '<=' | '>=' |
'&&' | '||';
unOp: '-' | '!';

```

```
ID: [a-zA-Z_][a-zA-Z0-9_]*;  
INT: [0-9]+;  
STRING: '"' .*? '"';  
WS: [ \t\r\n] -> skip;  
COMMENT: '//' ~[\r\n]* -> skip;
```

Результат

Тестування було проведено на таких програмах:

Код 1.1 — Безпомилковий код

```
fn main() {  
    println!("JEsus is LORD");  
    let y: bool = false;  
    // Here's a comment  
}
```

Код 1.2 — Помилковий код

```
fn main() {  
    let x = 0  
    let y = 3  
    bool start = false  
}
```

Результати:

ПИТАННЯ

Які завдання розв'язуються в процесі синтаксичного аналізу?

Основним завданням є перевірка структури тексту, чи відповідає вона правилам мови.

Що таке контекстно-вільна граматика?

Набір формальних правил, які визначають як будуються речення в мові.

Чим відрізняється та в чому полягають низхідний та висхідний синтаксичний аналіз?

Низхідний будує дерево від кореня до листів.

Висхідний будує від листя до кореня.